

BHTTP/1 Binary HTTP Protocol Specification

BHTTP/1 transfers files over TCP using one complete frame per message and persistent, sequential request/response exchanges. All integers are unsigned, big-endian; u8, u16, and u32 mean 1, 2, and 4 bytes. Offsets start at zero; lengths count bytes, not characters. There is no handshake, padding, terminator, or textual HTTP envelope. TCP reads must accumulate exactly the required bytes; writes must transmit every byte. MUST and MUST NOT state protocol requirements.

1. Fixed frame header

Every frame is the following 8 bytes, immediately followed by its payload.

Offset	Width	Field	Encoding
0	1	version/type	High nibble: version 1; low nibble: type
1	1	flags	MUST be 0 for REQUEST and RESPONSE
2	2	request ID	Correlates a request with its response
4	4	payload length	Payload bytes only; excludes these 8 bytes

Type **1** is REQUEST (client to server); type **2** is RESPONSE (server to client). Version-1 types **0 and 3..15** are unknown extensions: receivers MUST discard exactly the declared payload in chunks of at most 65,536 bytes, ignore flags/ID, send no reply, and continue with the next header. Zero-length unknown frames are valid. Unknown frames never complete an exchange. Unsupported versions and excessive lengths are fatal, including on unknown types.

2. Message payloads

REQUEST: method:u8 | path_length:u16 | path[path_length] | header_block

RESPONSE: status:u16 | header_block | body[remaining payload bytes]

REQUEST method MUST be **1 (GET)**. The path is strict UTF-8 and begins with /. Request headers are optional; their block MUST end exactly at the payload end. There is no request body; extra bytes are malformed. RESPONSE status is one final code in **200..599**. Its header count and entry lengths determine where the body starts: body length = payload length - 2 - encoded header-block length. Bodies are opaque bytes, including NUL, non-UTF-8 bytes, and byte-order marks.

3. Header encoding

Each block begins with count:u8, followed by exactly that many entries:

Numeric: name_id:u8 | value_length:u16 | value[value_length]

Literal: 0:u8 | name_length:u8 | name[name_length]
 | value_length:u16 | value[value_length]

The version-1 dictionary is fixed; IDs below are decimal.

ID	Name	ID	Name
1	content-type	6	server
2	content-length	7	connection
3	host	8	cache-control
4	accept	9	date
5	user-agent	10	last-modified

Selector 0 escapes to a literal name; selectors 11..255 are malformed. Names are case-sensitive lowercase ASCII matching [a-z][a-z0-9-]*. Values are strict UTF-8, may be empty, MUST NOT begin with ef bb bf (BOM), and MUST NOT contain U+0000..U+001F or U+007F. Do not trim whitespace. Senders MUST use numeric IDs for dictionary names and sort by decoded ASCII name. Receivers MUST accept any order and literal spellings of known names, reject duplicate decoded names, and accept/ignore well-formed unknown names.

4. Header semantics

Every response MUST include nonempty content-type, content-length equal to the body byte count, and connection equal to keep-alive or close. Content length is ASCII 0 or [1-9][0-9]*, with no leading zeros. If present in a request, content-length MUST be 0 and connection MUST be keep-alive. Other request headers are informational and cannot override framing.

The supplied client sends host: HOST:PORT, accept: */*, and user-agent: bcurl/1. Successful server responses also send server: bserve/1, cache-control: no-store, date, and last-modified; thus all ten dictionary names are transmitted. Dates use UTC YYYY-MM-DDTHH:MM:SSZ; receivers need only generic validation of informational values. File suffixes are compared case-insensitively: .html/.htm use text/html; charset=utf-8, .txt uses text/plain; charset=utf-8, and others use application/octet-stream.

5. Connection and request IDs

Requests use IDs **1..65535**; responses echo the ID. Clients start at 1, increment after a completed exchange, and wrap 65535 to 1. Servers accept any nonzero ID without enforcing sequence; reuse is allowed after completion. There is at most one outstanding request: wait for a complete response before the next request. No pipelining or multiplexing is supported.

The server keeps the connection open after normal and recoverable responses. `connection: close` announces closure after a fatal diagnostic. The one-resource CLI closes after its response. Each invocation chooses one resolved endpoint and attempts one TCP connection, without address fallback, retry, reconnect, or redirect. No idle timeout is mandated; local timeouts terminate the connection.

6. Errors and limits

Status	Meaning
200	Regular file served successfully
400	Malformed request or unsafe/invalid path
404	Missing path or non-file; includes directories, /, and trailing /
500	File/server failure, including an oversized file

A fully consumed, bounded version-1 REQUEST with nonzero ID but invalid flags, method, fields, UTF-8, headers, or path gets 400 with the echoed ID and `keep-alive`. Fatal cases are unsupported version, global/request oversize, request ID 0, known type in the wrong direction, or truncation: send a best-effort version-1 RESPONSE 400 with flags 0 and `close`, then `close` without draining or scanning for another frame. Echo an ID only from a complete version-1 REQUEST header with nonzero ID; otherwise use diagnostic ID 0. Clients accept ID 0 only for 400 with `close`; other mismatched IDs are errors.

EOF before any byte of a new header is a clean disconnect without a response; partial headers/payloads are truncation. Resets, write failures, and timeouts close the connection; diagnostic delivery cannot be guaranteed. Error bodies are short UTF-8 text with `text/plain; charset=utf-8`, without local paths/tracebacks. Clients validate responses, write body bytes to stdout, and return nonzero for 4xx/5xx or transport/protocol failures; valid 2xx/3xx return zero. Invalid replies close the connection, including invalid flags, version, direction, length, or ID.

Item	Accepted size/count
Global frame payload	At most 16,777,216 bytes (16 MiB)
REQUEST payload / entire header block	Each at most 16,384 bytes (16 KiB)
Response body	0..8,388,608 bytes (8 MiB)
Wire path / header value	1..4096 / 0..4096 bytes
Literal name / headers per block	1..255 bytes / 0..32 entries

Validate declared lengths before allocation and every field boundary before decoding. Read files as bounded binary snapshots before sending a response; content length describes actual bytes, not a prior file-size estimate.

7. Paths and file mapping

Percent-decode path bytes exactly once using %HH (ASCII hex digits), then strict UTF-8; malformed escapes/UTF-8 are 400 and + stays literal. Reject raw or decoded backslashes, controls U+0000..U+001F/U+007F, ?, and #; decoded paths must begin /. Reject repeated /, ./, .. components, components containing :<>"|*, and trailing dots/spaces. For device names, take the stem before its first dot, strip trailing ASCII spaces, and reject case-insensitively CON, PRN, AUX, NUL, CONIN\$, CONOUT\$, COM1..9, LPT1..9, and COM/LPT with ¹, ², ³.

Resolve components beneath the configured root, including symlinks/junctions; resolved escapes and loops are 400, missing paths 404, and only regular files are served. No automatic index/listing or case/Unicode normalization applies; file-name matching follows the host filesystem. The locally administered tree must not be adversarially replaced during resolution/opening.

8. Design rationale

The fixed header simplifies framing, while its explicit length makes extensions skippable. Packed 4-bit version/type fields leave small expansion namespaces; 8-bit flags preserve the compact layout. A 16-bit ID supports correlation without multiplexing; a 32-bit length leaves capacity beyond operational caps. Byte-sized counts/selectors/name lengths suffice for small metadata; 16-bit path/value lengths and status accommodate values beyond 255. The dictionary saves repeated names. The frame boundary already determines body length.